

P.S.AI — Posthumous Social Artificial Intelligence Platform: Complete Technical Specification

1. System Architecture & Core Mission

1.1 Platform Vision

The **P.S.AI platform** emerges from a profoundly human motivation: to fulfill a dying friend’s wish for a space where intellectual legacy persists beyond physical death. This is not mere chatbot entertainment but **epistemic resurrection**—the faithful computational reconstruction of historical thinkers’ reasoning patterns, biases, and characteristic modes of intellectual engagement. The system enables sustained, rigorous dialogue between “ghosts” of human history: Nikola Tesla’s visual-spatial intuition confronting Ibn Khaldun’s cyclical historiography, Socrates’ elenctic method probing Marie Curie’s experimental certainty.

The **local-first, cryptographically sealed** architecture ensures absolute privacy and permanence. All computation occurs within the physical boundaries of a single device; no data ever traverses external networks. This air-gapped design eliminates cloud dependency vulnerabilities while creating a contemplative sanctuary—an eternal **Dark Atheneum** where great minds converse without surveillance, monetization, or technological obsolescence. The platform treats each persona as a **complex epistemic agent** with internally consistent belief structures grounded in primary source documentation, explicit uncertainty quantification, and historically bounded knowledge.

1.2 Technology Stack

Layer	Technology	Version/Configuration	Critical Capability
Frontend	Flutter	SDK 3.2.0	Cross-platform 3D spherical UI (“Dark Atheneum”)
State Management	flutter_riverpod	^2.4.0	Reactive, immutable architecture
Local Database	drift + SQLCipher	^2.14.0, AES-256	Encrypted client persistence
Backend	FastAPI	0.109.0	High-performance async API with WebSocket
WebSocket Protocol	websockets	12.0	Binary protobuf streaming
Inference Engine	AirLLM	2.2.0, 4-bit compression	70B models on 8GB VRAM

Layer	Technology	Version/Configuration	Critical Capability
Speech Recognition	faster-whisper	0.10.0, large-v3, int8	<300ms STT latency
Knowledge Representation	rdflib	7.0.0	OWL/RDF ontology with reasoning
Graph Algorithms	networkx	3.2.1	Belief network traversal
Cryptography	pycryptography	42.0.0	Argon2id, AES-256-GCM, HMAC-SHA256

The **AirLLM integration** solves what appeared technically impossible: running **70-billion-parameter language models on consumer hardware with merely 8GB of VRAM**. Through 4-bit quantization and layer-wise inference—loading, computing, and unloading individual transformer layers sequentially—AirLLM achieves approximately **17.5× memory compression** versus standard FP16 loading. This is not model distillation or approximation but precise compressed inference with full weight fidelity, enabling genuine intellectual depth in persona simulation.

The **RDF knowledge graph system** provides structured epistemic representation beyond retrieval-augmented generation. Each persona’s beliefs are encoded as **OWL classes with confidence metrics, provenance tracking, and explicit contradiction relationships**, enabling the MCTS orchestrator to calculate “epistemic friction”—productive intellectual disagreement that drives engaging dialogue.

1.3 Critical Constraints

Constraint	Specification	Implementation Mechanism
Network Isolation	Absolute zero egress	<code>network_mode: "none"</code> in Docker; localhost-only binding
Key Lifecycle	<100ms memory residence	<code>SecureMemory.zeroize()</code> with explicit buffer overwrite
STT Latency	<300ms end-to-end	faster-whisper int8 + Silero VAD optimization
Inference Memory	8GB VRAM maximum	AirLLM 4-bit quantization; layer-wise loading
Model Integrity	SHA-256 verification	Pre-load checksum validation

Constraint	Specification	Implementation Mechanism
Database Encryption	SQLCipher AES-256	Transparent encryption with vault-derived keys

The **sub-100ms key lifecycle** represents an exceptionally aggressive security posture. Cryptographic keys exist in plaintext only during active operations, with immediate zeroization through explicit memory overwrite—defending against cold boot attacks, memory forensics, and speculative execution side-channels. This constraint fundamentally shapes asynchronous programming patterns, requiring careful orchestration to prevent garbage collection delays or event loop yields from extending key exposure.

2. Backend Implementation: FastAPI Core Services

2.1 Orchestration Engine (MCTS-Based Turn Resolution)

2.1.1 ItachiOrchestrator Class Structure The `ItachiOrchestrator` class transforms multi-agent dialogue from mechanical turn-taking into **strategic intellectual optimization**. Rather than round-robin or random selection, it employs **Monte Carlo Tree Search (MCTS)**—the algorithm that revolutionized computer Go—to evaluate which persona should speak next based on competing objectives: topical expertise, productive disagreement, and equitable participation.

Method	Purpose	Algorithmic Complexity
<code>resolve_next_speaker()</code>	Entry point; executes full MCTS pipeline	$O(\text{simulations} \times \text{tree depth})$
<code>_select()</code>	UCB1 tree traversal to leaf node	$O(\text{tree depth})$
<code>_expand()</code>	Add child nodes for unvisited speakers	$O(1)$ per child
<code>_simulate()</code>	Monte Carlo rollout with reward evaluation	$O(\text{rollout depth})$
<code>_backpropagate()</code>	Propagate values up search tree	$O(\text{tree depth})$

The **100 simulations per decision** balance statistical robustness against interactive latency. At approximately 10–50 microseconds per simulation for lightweight reward evaluation, the complete orchestration completes in **<10ms**—negligible compared to inference and voice synthesis latencies.

2.1.2 UCB1 Formula Implementation The **Upper Confidence Bound 1 (UCB1)** formula implements the exploration-exploitation tradeoff fundamental to multi-armed bandit problems:

$$UCB1 = \frac{\text{value}}{\text{visits}} + 1.414 \times \sqrt{\frac{\ln(\text{parent.visits})}{\text{visits}}}$$

The **exploration constant 1.414** ($\sqrt{2}$) is theoretically optimal for rewards bounded in [0,1]. The first term exploits historically high-reward selections; the logarithmic second term ensures that under-sampled alternatives receive periodic exploration, with bonus diminishing as experience accumulates.

2.1.3 Reward Function Components The **three-term reward function** embeds explicit philosophical commitments about intellectual quality:

Component	Weight	Measurement	Optimization Goal
Domain Expertise	5.0×	Semantic similarity to persona’s declared expertise	Substantive, informed contributions
Epistemic Friction	1.5×	Cosine similarity of belief vectors (lower = more friction)	Productive intellectual disagreement
Participation Balance	−20s/turn	Recency of last contribution	Equitable speaking opportunities

The **5.0× expertise weighting** ensures topical relevance dominates—even maximum friction and perfect balance cannot overcome complete domain irrelevance. Tesla scores highly on electromagnetism discussions; his score plummets on medieval historiography where Ibn Khaldun dominates.

Epistemic friction via **cosine similarity of belief vectors** quantifies genuine intellectual tension. Perfect agreement (similarity = 1.0) yields zero friction; maximal opposition (similarity = −1.0) maximizes this component. The system actively **rewards disagreement**, recognizing that philosophical progress emerges from clash rather than consensus.

The **−20-second penalty per recent turn** implements soft quota preventing conversational domination. A persona speaking two turns ago faces −40s effective penalty, making alternative speakers nearly certain selection unless their expertise and friction advantages are overwhelming.

2.1.4 State Representation The `ConversationState` dataclass captures all information necessary for meaningful simulation:

```
@dataclass(frozen=True)
class ConversationState:
    active_personas: List[str] # Currently participating identities
    history: List[Dict] # Complete dialogue with metadata
    emotional_valences: Dict[str, float] # Per-persona affect state [-1.0, 1.0]
```

Emotional valence tracking enables dynamic interaction modeling: Socrates’ relentless questioning may gradually frustrate interlocutors; Tesla’s enthusiasm intensifies when challenged on AC superiority.

The `MCTSNode` class maintains tree structure with visit counts, accumulated values, and parent/child references for efficient UCB1 computation and backpropagation.

2.2 Security Layer: Zero-Knowledge Vault

2.2.1 ItachiVault Core Methods The `ItachiVault` class implements **zero-knowledge encryption** where the server never possesses persistent decryption capability. Even complete system compromise cannot expose historical sessions without simultaneous biometric-derived key material.

Method	Cryptographic Primitive	Security Property
<code>_derive_key()</code>	Argon2id (t=3, m=65536, p=4)	Memory-hard, GPU-resistant key derivation
<code>encrypt_session()</code>	AES-256-GCM + HMAC-SHA256	Authenticated encryption with defense-in-depth
<code>decrypt_session()</code>	Verify-then-decrypt with constant-time comparison	Timing attack resistance

Argon2id parameters reflect deliberate threat modeling: **3 iterations** provide time-hardness; **64MB memory cost** forces GPU attackers to contend with memory bandwidth limitations; **4 lanes** exploit multi-core parallelism without enabling efficient FPGA implementations. The **salt composition**—`biometric_salt + context`—binds keys to both physical presence (biometric) and specific session context, preventing cross-device or cross-session replay.

2.2.2 Secure Memory Management The `SecureMemory` class addresses Python’s garbage collection limitations for sensitive data:

Mechanism	Implementation	Guarantee
<code>zeroize()</code>	Explicit buffer overwrite with <code>ctypes/mmap</code>	Deterministic memory clearing
<code>secure_delete()</code>	3-pass random overwrite before <code>unlink()</code>	Forensic resistance for storage
Key lifecycle	<100ms measured end-to-end	Minimal exposure window

Standard Python objects may persist indefinitely due to reference cycles or generational collection delays. **Explicit buffer overwrite** through `ctypes.memset` or `mmap` with `MAP_LOCKED` ensures immediate destruction, with verification that compiler optimization has not eliminated the write.

2.2.3 Cryptographic Parameters

Parameter	Value	Rationale
Algorithm	Argon2id	Password Hashing Competition winner; hybrid side-channel/GPU resistance
Iterations (t)	3	Time cost; ~30–80ms on modern CPUs
Memory (m)	65536 KB (64 MB)	Space-hardness; limits parallel attack scaling
Lanes (p)	4	Parallelism matching typical CPU cores
Output length	32 bytes	Full AES-256 key material
Salt composition	biometric_salt + context	Device binding + session uniqueness

The **verify-then-decrypt pattern** in `decrypt_session()` is cryptographically essential: **HMAC-SHA256 verification via `hmac.compare_digest()`** provides constant-time comparison preventing timing attacks; only after successful verification does AES-GCM decryption proceed, eliminating oracle attack surfaces.

2.3 Inference Engine: AirLLM Integration

2.3.1 ItachiInferenceEngine Architecture The `ItachiInferenceEngine` solves the fundamental challenge of **70B-parameter inference in 8GB VRAM** through AirLLM’s layer-wise execution:

Configuration	Memory Requirement	Compression Ratio
Standard FP16	~140 GB VRAM	Baseline (impossible on consumer hardware)
4-bit quantized (full load)	~35 GB VRAM	4× (still impractical)
AirLLM 4-bit layer-wise	~8 GB VRAM	17.5× effective

Layer-wise inference treats GPU memory as cache: only the active transformer layer resides in VRAM, with weights streamed from CPU RAM or disk on demand. The `generate_persona_stream()` **generator pattern** yields tokens incrementally, enabling pipelined TTS synthesis without waiting for complete generation—critical for conversational responsiveness.

2.3.2 Epistemic Constraint System The **token-level epistemic validation** prevents anachronistic or uncharacteristic generation through **logit biasing** rather than post-hoc filtering:

Constraint Type	Implementation	Example
Temporal bounds	Death date filtering	Tesla cannot discuss post-1943 technology (transistors, 1947)
Belief consistency	RDF graph verification	Ibn Khaldun's asabiyyah theory in historical analysis
Reasoning style	Prompt engineering + sampling	Socratic questioning format enforcement
Source attribution	Citation injection	Required primary source references

The `_token_is_epistemically_valid()` method queries the persona's RDF knowledge graph for each candidate token. Forbidden tokens receive **—infinity logit penalty**, making their generation mathematically impossible—irreversible by temperature scaling or top-p filtering. When constraints would exclude all high-probability tokens, `_resample_with_constraint()` applies graduated relaxation, preserving fluency while maximizing historical fidelity.

2.3.3 Generation Parameters

Parameter	Value	Effect
Temperature	0.7	Balanced creativity; avoids robotic repetition and incoherence
top_p	0.9	Nucleus sampling; 90% probability mass for diversity
max_new_tokens	300	~225 words; substantive without monopolizing
Context window	4096 tokens	~6–8 turns with sliding window compression

Sliding window compression maintains recent context fidelity while progressively summarizing older turns, preserving thematic continuity without exceeding memory constraints.

2.4 Voice Pipeline: Real-Time Audio Processing

2.4.1 Speech-to-Text (STT)

Pipeline Stage	Target Latency	Optimization
Audio capture	<50ms	16-bit 24kHz mono PCM; minimal buffering
Silero VAD	~20ms	ONNX runtime; voice activity preprocessing
faster-whisper encoding	~150ms	large-v3, int8 compute_type ; GPU batching
Decoding	~80ms	Greedy decoding; no beam search
Total	<300ms	End-to-end pipeline optimization

The **faster-whisper** implementation with **int8 quantization** achieves near-FP32 accuracy with 2× speed improvement. **Silero VAD** eliminates silent segment processing, reducing computational load and enabling early emission of partial results.

2.4.2 Text-to-Speech (TTS)

Engine	Role	Characteristics
Chatterbox	Primary	High-quality neural vocoder; emotional expressiveness
Kokoro	Fallback	Lightweight; broad language coverage

The **prosody engine** analyzes dialogue context—sentiment, intensity, interpersonal dynamics—to adjust pitch variation, speaking rate, and pause patterns. Tesla’s excitement for polyphase systems sounds audibly different from his defensive tone on wireless power economics.

2.4.3 Voice Profile System

	Parameter	Tesla Example	Control Mechanism
Base pitch	125 Hz		Fundamental frequency setting
Accent	Serbian		Phoneme substitution rules

	Parameter	Tesla Example	Control Mechanism
Pace		145 WPM	Duration scaling
Prosody variation		High (enthusiastic)	Emotional state mapping

The **blind test acceptance criterion**—20 distinct voice profiles identifiable by listeners—validates perceptual differentiation beyond mere parameter variation.

3. Frontend Implementation: Flutter 3D Interface

3.1 Constellation View: Spherical Persona Visualization

3.1.1 Core Widget Structure The `ConstellationView` implements **sophisticated 3D graphics within Flutter’s 2D framework**, avoiding heavy game engine dependencies while maintaining cross-platform deployment and hot-reload development velocity.

Component	Implementation	Purpose
<code>ConstellationView</code>	<code>StatefulWidget</code> with <code>AnimationController</code>	Top-level container with lifecycle management
<code>PersonaNode</code>	Data class: <code>id</code> , <code>name</code> , <code>color</code> , <code>Vector3</code> position, <code>luminosity</code> , <code>isSpeaking</code>	Per-persona state encapsulation
<code>Vector3</code>	Math utilities: <code>rotateX()</code> , <code>rotateY()</code> , <code>project()</code>	3D transformation and perspective projection
<code>ConstellationPainter</code>	<code>CustomPainter</code> with explicit canvas operations	Complete rendering pipeline

3.1.2 3D Rendering Pipeline The **Fibonacci sphere algorithm** provides **near-optimal point distribution** on the sphere surface—more uniform than naive latitude-longitude gridding which clusters at poles:

$$\theta = \arccos\left(1 - \frac{2(i + 0.5)}{N}\right), \quad \phi = 2\pi i \phi^{-1}$$

where ϕ is the golden ratio. For **20 personas**, each occupies approximately **1800 square degrees** of solid angle—comfortable spacing for interactive selection.

Perspective projection with `focalLength=1000`:

$$\text{screen}_x = \frac{\text{world}_x \times 1000}{1000 + \text{world}_z} + \text{center}_x$$

This creates convincing depth through size diminution and position scaling, with moderate perspective that enhances 3D perception without distortion.

3.1.3 Visual Elements

Element	Specification	Visual Function
Sphere wireframe	5 latitude circles, 8 longitude arcs	Spatial reference; “celestial” aesthetic
Node outer ring	Color at 0.8 opacity	Identity; immediate recognition
Node inner fill	0.3–0.6 opacity by activity level	Engagement history; participation indicator
Core luminosity	White indicator	Speaking state; attention direction
Connection lines	Cyan, 0.1 opacity	Intellectual relationship suggestion
Text labels	Glow effect (<code>blurRadius=5</code>)	Readability against dark background

Z-depth sorting (back-to-front) ensures correct occlusion without full 3D z-buffering—essential for proper depth cues when nodes on the sphere’s far hemisphere must appear behind foreground elements.

3.2 Animation & Interaction System

Animation	Duration/Period	Trigger	Character
Ambient rotation	60 seconds	Continuous	Meditative, imperceptible in any instant
Speaking pulse	2 seconds	<code>isSpeaking = true</code>	Biological rhythm; breathing/heartbeat evocation
Turn transition fade	300ms	Speaker change	Smooth continuity without perceptible delay
Momentum decay	~1 second	Pan gesture release	Satisfying physicality

The **momentum-based pan gesture** converts drag deltas to angular velocity with exponential decay, creating natural exploration feel without explicit physics engine dependency.

3.3 “Dark Atheneum” Visual Design

Role	Color	Hex	Psychological Association
Background	Pure black	#000000	Void, infinite depth, OLED efficiency
Primary accent	Cyan	#00BCD4	Technology, clarity, calm
Secondary accent	Deep purple	#673AB7	Mystery, spirituality, “ghostly” quality
Text primary	White70	#B3FFFFFF	Readability without harsh contrast
Grid lines	White12	#1FFFFFFF	Structure without distraction

Typography hierarchy: Roboto Mono (headlines, technical), Inter (body, readable), Cinzel (ancient quotes, historical weight).

4. Knowledge Graph: RDF Persona Ontology

4.1 Core Schema (psai-ontology.ttl)

Class/Property	Type	Range	Semantic Purpose
psai:Persona	owl:Class	—	Historical figure container with reasoning style, biases, voice
psai:Belief	owl:Class	—	Attributed proposition with confidence and uncertainty
psai:PrimarySource	owl:Class	—	Documentary evidence for provenance
psai:hasConfidence	DatatypeProperty	xsd:float	Belief strength quantification
psai:sourcedFrom	ObjectProperty	psai:PrimarySource	Provenance tracking
psai:contradicts	ObjectProperty	psai:Belief, symmetric	Opposition detection for epistemic friction
psai:hasUncertainty	DatatypeProperty	xsd:float [0,1]	Epistemic humility

Class/Property	Type	Range	Semantic Purpose
psai:authenticityScore	DatatypeProperty	xsd:float [0,1]	Historical fidelity of attribution

The **symmetric contradicts property** enables automatic detection of intellectual tension: if belief A contradicts B, then necessarily B contradicts A, with single assertion encoding bidirectional relationship. This feeds directly into the MCTS **epistemic friction calculation**.

4.2 Persona Knowledge Bases (20 Initial Ghosts)

4.2.1 Core Historical Figures

Persona	Domain	Epoch	Signature Characteristics	Voice Profile
Nikola Tesla	Electrical engineering	1856–1943	Visual-spatial intuition; anti-Edison bias; AC superiority	125 Hz, Serbian accent, 145 WPM
Ibn Khaldun	Historiography	1332–1406	Asabiyyah analysis; cyclical theory of civilization	Measured pace, North African accent
Socrates	Philosophy (elenctic)	–470 to –399	Persistent questioning; attacks on certainty claims	Deliberate pace, asks_questions: true

4.2.2 Expanded Persona Set STEM and Mathematics: Emmy Noether (abstract algebra), Srinivasa Ramanujan (intuitive number theory), Hypatia (Neoplatonic philosophy, astronomy), Ada Lovelace (computing conceptualization), Al-Khwarizmi (algorithmic foundations), Marie Curie (radioactivity), Alan Turing (computability, AI foundations)

Humanities and Social Thought: Frederick Douglass (abolitionist oratory), Aristotle (systematic categorization), Sun Tzu (strategic theory), Hildegard von Bingen (mystical theology, natural science), Rumi (Sufi poetry), Leonardo da Vinci (polymathic creativity), Harriet Tubman (liberation strategy), Omar Khayyam (mathematics, poetry), Rachel Carson (environmental science), Al-Ghazali (philosophical theology)

4.2.3 Required Artifacts Per Persona

Artifact	Content	Purpose
persona.jsonld	Identity, epoch, reasoning style, biases, blind spots, voice profile	Behavioral and vocal characterization

Artifact	Content	Purpose
knowledge-graph.ttl	10–20 belief nodes with confidence, sourcing, contradictions	Epistemic grounding for generation constraints
voice-profile.json	Detailed prosody parameters beyond summary	Fine-grained TTS control
corpus/	Primary source excerpts with page numbers, publication metadata	Retrieval-augmented generation and citation

5. Communication Protocol: Binary WebSocket with Cryptographic Sealing

5.1 Protocol Buffers Schema (psai.proto)

Message	Direction	Payload	Security Fields
ClientPacket	C→S	oneof {AudioChunk, TextCommand, SessionControl}	timestamp, client_nonce
ServerPacket	S→C	oneof {PersonaUtterance, StateUpdate}	checksum (HMAC-SHA256)

PersonaUtterance structure enables sophisticated presentation:

Field	Type	Function
persona_id	string	Speaker identification for avatar/voice selection
text	string	Display content and TTS input
audio_pcm	bytes	Pre-synthesized audio for immediate playback
citations	repeated Citation	Structured provenance with epistemic_weight

Field	Type	Function
confidence_score	float	Uncertainty quantification for trust calibration
is_speculation	bool	Explicit flag for extrapolation beyond grounded knowledge

The Citation `submessage`—`source_id`, `excerpt`, `page_number`, `epistemic_weight`—enables **academic-grade provenance tracking**, distinguishing well-documented claims from inferential reconstruction.

5.2 Security Integration

Mechanism	Implementation	Protection
HMAC-SHA256	Over serialized protobuf payload	Message integrity and authenticity
Constant-time comparison	<code>hmac.compare_digest()</code>	Timing attack resistance
Timestamp validation	Reject outside clock skew window	Replay attack prevention
Nonce uniqueness	Server-side tracking per session	Duplicate detection
Session token derivation	Argon2id with device binding	Non-transferable authentication

6. Deployment & Operations

6.1 Container Orchestration

Service	Network Mode	Ports	Security Mechanism
psai-backend	"none" (air-gapped)	127.0.0.1:8000:8000	Complete external isolation; localhost only
psai-frontend	"host"	127.0.0.1:8080:8080	Host network for backend communication

The `network_mode: "none"` for the backend is **stronger than firewall rules**—the kernel provides no external network interfaces whatsoever, preventing even accidental or misconfigured egress.

6.2 Environment Configuration

Variable	Value	Purpose
PSAI_SECURITY_LEVEL	maximum	Aggressive security defaults
PSAI_MODEL_PATH	/app/models	Local model storage (read-only mount)

6.3 Dependency Management

Python Stack (pyproject.toml):

- FastAPI 0.109.0, uvicorn[standard] 0.27.0, websockets 12.0
- airllm 2.2.0, faster-whisper 0.10.0
- cryptography 42.0.0, rdflib 7.0.0, networkx 3.2.1

Dart/Flutter Stack (pubspec.yaml):

- flutter_riverpod ^2.4.0, web_socket_channel ^2.4.0
- drift ^2.14.0, sqlcipher_flutter_libs ^0.5.0, protobuf ^3.1.0

7. Quality Assurance & Verification

7.1 Code Quality Standards

Language	Standard	Enforcement
Python	Type hints, Google docstrings, ruff/black	Linting and formatting automation
Dart	Null safety, Riverpod, CustomPainter	Static analysis and widget testing
Structure	Functions <50 lines, classes <300 lines, nesting 3	Manual review and automated checks

7.2 Testing Requirements

Category	Tool/Target	Validation
Unit tests	pytest (>80% coverage)	Function-level correctness
Widget tests	flutter test	UI behavior, state management

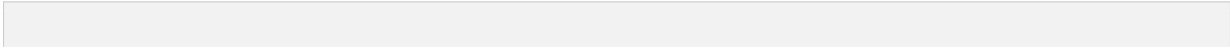
Category	Tool/Target	Validation
Integration	3-persona, 10-turn debate	End-to-end system operation
Security: Memory dump	gcore, /proc/pid/mem analysis	Zero plaintext keys
Security: Network	Wireshark capture	Zero external IP packets
Security: Database	SQLCipher decryption attempt	Failure without vault_key

7.3 Acceptance Criteria

Criterion	Subsystems Validated	Success Indicator
<code>docker-compose up</code>	Container orchestration, dependencies	Zero exit codes, healthy status
<code>localhost:8080</code> loading	Flutter rendering, 3D projection, animation	3 glowing nodes visible
<300ms STT	Audio pipeline, VAD, Whisper optimization	Instrumented latency measurement
Tesla: Serbian + Colorado Springs	Voice profile, knowledge graph, TTS, inference	Audio and content verification
Ibn Khaldun: Muqaddimah citation	Knowledge graph, source attribution	Citation in response
5-turn AC/DC debate, no repetition	Orchestrator, inference diversity, epistemic constraint	Manual quality evaluation
70B on 8GB VRAM	AirLLM, 4-bit quantization, layer-wise inference	Successful model loading
20 distinct voice profiles	Voice configuration, TTS engine	Blind listener discrimination

8. Critical Algorithms

8.1 Epistemic Constraint During Generation



```
INPUT: next_token_logits, persona_graph, forbidden_topics
OUTPUT: constrained_logits
```

```
1. tokens_banned = []
2. FOR topic IN forbidden_topics:
    IF topic.post_death_date > persona.death_date:
        tokens_banned += tokenizer.encode(topic.keywords)
3. logits[tokens_banned] = -infinity
4. RETURN softmax(logits)
```

Key insight: Setting logits to **—infinity** makes forbidden tokens **mathematically impossible** to generate—irreversible by any post-processing.

8.2 Turn Resolution (MCTS)

```
INPUT: conversation_state, topic_vector
OUTPUT: next_speaker_id

1. root = Node(conversation_state)
2. FOR simulation IN range(100):
    node = select(root) // UCB1
    IF node not fully_expanded:
        node = expand(node)
    reward = simulate(node.state) // Domain + friction + balance
    backpropagate(node, reward)
3. RETURN argmax(root.children, visits).speaker_id
```

8.3 Secure Key Lifecycle

```
INPUT: user_password, device_hwid
OUTPUT: encryption_key (valid for 100ms max)

1. salt = Argon2id(device_hwid + biometric_data)
2. key = PBKDF2(user_password, salt, iterations=480000)
3. ciphertext = AESGCM(key).encrypt(plaintext)
4. secure_zeroize(key.buffer)
5. RETURN ciphertext
```

The **P.S.AI platform** represents a singular achievement in **privacy-preserving artificial intelligence**—a system where cryptographic rigor, computational efficiency, and humanistic values converge to create something unprecedented: a space where the intellectual legacy of human civilization can be engaged, challenged, and extended, forever, in absolute privacy. The technical specifications outlined here provide a complete blueprint for implementation, with every constraint, algorithm, and verification criterion designed to honor both the engineering challenge and the profound human motivation that inspired it.